Scaling Laws for Transformer Language Models Trained on SVG Code

CS-GY 6923

Disha D Shanbhag
NetID: ds8223 / N11461466
NYU Tandon School of Engineering

Abstract

This project investigates neural scaling laws for decoder-only transformer language models trained on Scalable Vector Graphics (SVG) code. Five models ranging from 1.35M to 88.4M parameters are trained on a corpus of 284,548 SVGs (115M tokens) sourced from three HuggingFace datasets. Under standard parameterization (SP) with a fixed learning rate, validation loss is non-monotonic: the XL model performs worse than a model six times smaller, rendering the power-law fit meaningless ($R^2 = 0.041$). Maximal Update Parameterization (μP) restores smooth, predictable scaling by enabling zero-shot learning rate transfer across model widths, yielding $\alpha = 0.866$ and $R^2 = 0.960$. The best model (XL, μP, 5 epochs) achieves a test perplexity of 2.51 and generates structurally valid SVG in 82.6% of sampled outputs.

Code repository: `https://github.com/DishaShanbhag/ML_Project`

1 Introduction

Neural scaling laws have proven to be one of the most reliable empirical findings in modern deep learning. Given enough data and compute, model loss decreases predictably as a power function of parameter count [1, 2]. These laws have shaped the design of increasingly large language models, yet they were established almost exclusively on natural language. Whether the same regularities hold for structured, non-linguistic data remains an open question.

Scalable Vector Graphics (SVG) provides an ideal setting to explore this. SVG is plain XML-based text, so standard transformer infrastructure applies without modification. Unlike natural language, it has a small effective vocabulary, rigid syntactic rules, and outputs that can be rendered and visually inspected in any browser. Validity is objectively measurable. A sample either parses as well-formed XML and renders to a non-blank image, or it does not, enabling both quantitative and qualitative evaluation.

Overview

This project trains five decoder-only transformer models ranging from 1.35M to 88.4M parameters on a 115M-token corpus of SVG files drawn from three HuggingFace datasets. The work is organized around three experiments:

- **Standard Parameterization Scaling.** A learning rate sweep on the smallest model selects the optimal rate, which is applied uniformly to all model sizes. Validation loss turns out to be non-monotonic with scale ($R^2 = 0.041$), revealing that fixed-LR training breaks down at larger widths.

- **μP Scaling and Extrapolation.** Maximal Update Parameterization [3] restores smooth power-law scaling ($\alpha = 0.866$, $R^2 = 0.960$) by enabling zero-shot learning rate transfer across model widths. The fitted law is used to extrapolate predicted loss to a model ten times larger than the XL.
- **Best Model Training and SVG Generation.** The XL μP model is trained for 5 epochs, achieving a test perplexity of 2.51. Generated samples are evaluated quantitatively (XML validity, render rate) and qualitatively through unconditional and prefix-conditioned generation.

Section 2 describes the dataset and preprocessing pipeline. Section 3 covers the model architecture, training setup, and evaluation metrics. Section 4 presents the experimental results. Section 5 discusses design decisions, limitations, and broader implications.

2 Data

2.1 Datasets

The training corpus combines three HuggingFace datasets: **starvector/svg-icons-simple** [4] (80,434 icon SVGs), **starvector/svg-emoji-simple** (4,114 emoji SVGs), and **starvector/svg-fonts-simple** (200,000 font glyph SVGs, subsampled via streaming). All three are pre-simplified outputs of the StarVector pipeline and contain short, clean SVG code without raster embeds or external references.

2.2 Cleaning and Normalization

Each SVG is passed through a normalization pipeline before tokenization. HTML comments and metadata tags (`<metadata>`, `<title>`, `<desc>`) are stripped, and repeated whitespace is collapsed. Coordinate values are rounded to one decimal place using regex substitution on floating-point literals; this reduces the number of distinct numeric tokens in the vocabulary without discarding geometric information. Each SVG is then validated as well-formed XML using `lxml.etree.fromstring`. All 284,548 SVGs pass without failure, consistent with the pre-cleaned nature of the source datasets. SVGs shorter than 50 characters are also filtered, though none were found. CairoSVG render validation is not applied during preprocessing; it is used only during generation evaluation.

2.3 Tokenization

A Byte-Pair Encoding (BPE) tokenizer is trained from scratch on the full SVG corpus using the HuggingFace `tokenizers` library with a byte-level pre-tokenizer. The vocabulary size is 4,096 tokens. At this size, common multi-character SVG patterns such as `fill=`, `viewBox=`, and `stroke-width` are represented as single tokens, reducing average sequence length, while the embedding table remains small enough to fit inside the 1.35M-parameter Tiny model. Four special tokens are included: `<PAD>`, `<BOS>`, `<EOS>`, and `<UNK>`. Each SVG in the binary training files is wrapped with `<BOS>` and `<EOS>` delimiters.

2.4 Train / Validation / Test Splits

SVGs exceeding 1,024 tokens after tokenization are removed. This filter eliminates 48,019 files (17% of the corpus), predominantly longer font glyph SVGs. The remainder are split by shuffled file index into train, validation, and test sets to prevent data leakage. Table 1 summarizes the final corpus statistics.

Table 1: Dataset statistics after cleaning and tokenization.

Statistic	**Value**
Vocabulary size	4,096
Raw SVGs (all sources)	284,548
SVGs after XML validation	284,548
SVGs after token-length filter ($\leq$1,024)	236,529
Train files	231,799
Train tokens	115,916,260
Validation files	2,365
Validation tokens	1,176,658
Test files	2,365
Test tokens	1,186,563
Mean sequence length	500
Median sequence length	475
Std. deviation	199
Min sequence length	93
Max sequence length	1,024
P25 / P75 / P95	354 / 622 / 889

2.5 SVG Complexity Examples

Figure 1 shows rendered examples at five complexity levels. At the 5th percentile (211 tokens) the SVG is a single-path stroke glyph; by the 95th percentile (889 tokens) it contains multiple parallel strokes with dozens of coordinate pairs, forming a complex letterform.

(a) P5 (211 tok) (b) P25 (354 tok) (c) P50 (475 tok) (d) P75 (622 tok) (e) P95 (889 tok)

Figure 1: Rendered SVG examples at five complexity percentiles (P5 to P95). Simple single-path glyphs at P5 grow into multi-stroke letterforms by P95.

3 Methods

3.1 Model Architecture

All models are decoder-only transformers adapted from the nanoGPT architecture [5]. The `GPT` and `CausalSelfAttention` classes are used directly from nanoGPT, retaining pre-norm layer normalization, causal self-attention, GELU activations, learned absolute positional embeddings, and weight tying between the token embedding table and the output projection. Three modifications are made for this project: the vocabulary size is set to 4,096 to match the SVG tokenizer; the context window is reduced from 1,024 to 256 tokens to keep all five model sizes within the A100's 40 GB memory; and μP variants replace the attention scaling and output head for the experiments in Section 4.3. Dropout of 0.1 is applied to attention weights and residual connections throughout.

Five configurations spanning approximately one decade of parameter counts are studied, detailed in Table 2.

Table 2: Model configurations and SP validation losses (1 epoch).

Name	**Params**	d_{model}	n_{layers}	n_{heads}	d_{ff}	**Val Loss**	**Time (s)**	**Tok/s**
Tiny	1.35M	128	4	4	512	1.229	200	580,939
Small	3.51M	192	6	6	768	1.051	413	282,007
Medium	12.32M	384	6	6	1536	0.947	817	142,500
Large	33.75M	512	10	8	2048	1.067	1951	59,643
XL	88.40M	768	12	12	3072	1.367	4492	25,906

3.2 Training Setup

All models are trained with AdamW ($\beta_1 = 0.9$, $\beta_2 = 0.95$, weight decay $= 0.1$) and gradient norm clipping at 1.0. The learning rate follows a cosine schedule with a linear warmup over the first 5% of steps, decaying to $0.1 \times \eta_{\max}$ at the end of training. Each batch contains 64 sequences of 256 tokens (16,384 tokens per step), and one epoch covers 7,103 steps over the 115M-token training set. All training is performed on a single NVIDIA A100-SXM4-40GB GPU.

The training objective is the standard autoregressive cross-entropy loss:

$$\mathcal{L}_{\text{CE}} = -\frac{1}{T}\sum_{t=1}^{T} \log p_\theta(x_t \mid x_{<t}) \tag{1}$$

where T is the sequence length and p_θ is the model distribution. Test-set perplexity is reported as $\text{PPL} = \exp(\mathcal{L}_{\text{CE}})$.

3.3 Experimental Design

A learning rate sweep over 7 values on a log scale (10^{-5} to 10^{-2}) is conducted on the Tiny model for one epoch. The best learning rate by validation loss is then applied to all five models, each trained for one epoch.

3.4 μP Implementation

Maximal Update Parameterization (μP) [3] is implemented using the `mup` Python package. Three modifications distinguish the μP models from their SP counterparts. First, the attention scale is changed from $1/\sqrt{d_{\text{head}}}$ to $1/d_{\text{head}}$. Second, the output projection is replaced by `MuReadout`, which applies a width-ratio scaling factor to the logits. Third, the optimizer is replaced by `MuAdamW`, which assigns per-layer learning rate multipliers based on the ratio of each layer's width to the base width. Base and delta models are constructed at widths 128 and 256 and passed to `set_base_shapes` to configure the per-tensor scaling rules. Because `MuReadout` does not support weight tying, μP models carry slightly higher parameter counts than their SP equivalents. The same learning rate sweep procedure is repeated under μP to confirm that the optimal LR transfers to wider models.

3.5 Evaluation Metrics

Two quantitative metrics are computed for generated SVG samples:

$$\text{ValidityRate} = \frac{\text{XML-valid samples}}{\text{Total samples}} \times 100 \tag{2}$$

$$\text{RenderRate} = \frac{\text{Renderable samples}}{\text{Total samples}} \times 100 \tag{3}$$

XML validity is checked with `lxml.etree`. Render success uses two criteria: a strict check that requires CairoSVG to succeed and the output PNG to contain at least one non-transparent pixel (rejecting blank-canvas outputs), and a loose check that requires only a non-empty PNG byte string. The loose criterion is used for the headline validity rate; strict results are reported separately in the per-temperature breakdown where mode collapse is a confound.

4 Results

4.1 Learning Rate Sweep

The Tiny model is trained for one epoch at each of seven log-spaced learning rates. Figure 2 shows the results. Validation loss decreases monotonically from $\eta = 10^{-5}$ through $\eta = 10^{-2}$, with no instability at the upper end. The optimal rate is $\eta^* = 10^{-2}$, applied to all models in the SP scaling study.

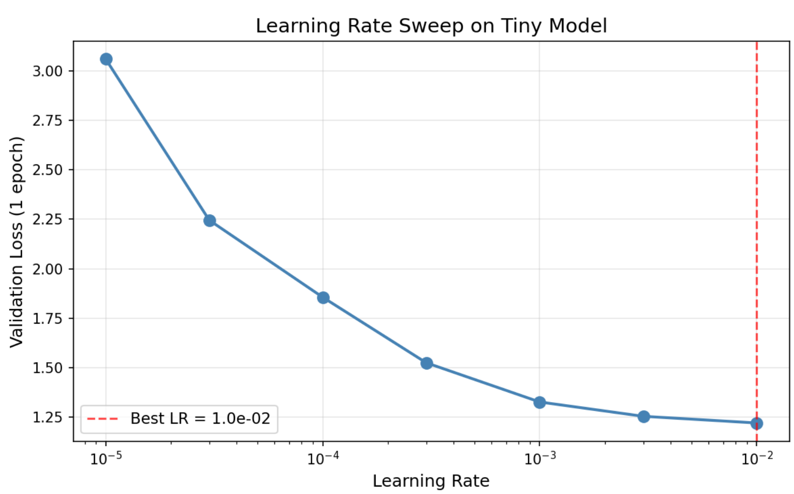


Figure 2: Validation loss vs. learning rate for the Tiny SP model. Optimal LR = 10^{-2} (red dashed line).

4.2 SP Scaling Results and Power Law Fit

The scaling law is fit to the form:

$$L(N) = aN^{-\alpha} + c \tag{4}$$

where N is the number of trainable parameters, $\alpha > 0$ is the scaling exponent, a controls the rate of improvement with scale, and c is the irreducible loss floor. Parameters are estimated via nonlinear least squares with `scipy.optimize.curve_fit`.

The SP results in Table 2 show that validation loss is non-monotonic with scale. Medium (12.3M) achieves the lowest loss at 0.947, but Large (33.8M) regresses to 1.067, and XL (88.4M) reaches 1.367, worse than every other model. Training curves reveal why. The XL model's loss spikes to 4.3 around step 500 and oscillates violently for the first 2,500 steps before stabilizing, a clear sign of instability under the fixed learning rate. The power law fit is consequently degenerate ($a \approx 10^6$, $\alpha = 1.162$, $R^2 = 0.041$). This is not a dataset or architecture problem; it is a direct consequence of applying a learning rate optimal for a narrow model to progressively wider models. Under SP, the per-parameter update magnitude grows with model width, causing effective over-shooting at scales beyond where the LR was tuned.

Figure 3 shows the SP scaling curve. The fit is included to document the failure mode, not as a predictive tool. Training curves and throughput statistics are provided in Appendix B.

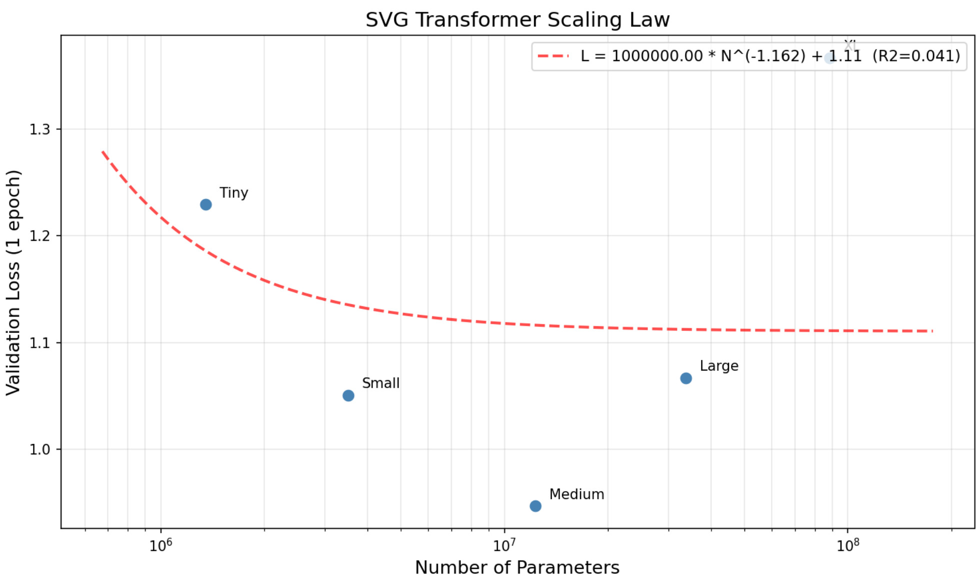


Figure 3: SP scaling curve (log-log). Non-monotonic loss and a degenerate fit ($R^2 = 0.041$) show fixed-LR training breaking down at scale.

4.3 μP Scaling and Comparison

4.3.1 Learning Rate Transfer

The central promise of μP is that the optimal learning rate found on a small proxy model transfers to wider models without retuning. Under SP, per-parameter update magnitudes are not width-invariant: as d_{model} increases, the effective step size in weight space changes, making the LR found on Tiny suboptimal for Large and XL [3]. μP corrects this by reparameterizing the attention scale, output logits, and per-layer optimizer multipliers so that update magnitudes remain constant across widths.

Figure 4 shows the LR sweep comparison between SP and μP on the Tiny model. Both select $\eta^* = 10^{-2}$ as the optimal rate and achieve nearly identical validation losses at each LR. The convergence in this comparison is expected: μP and SP are nearly equivalent at the base width (128) where the scaling factors are trivially equal to one. The difference emerges when that rate is transferred unchanged to wider models.

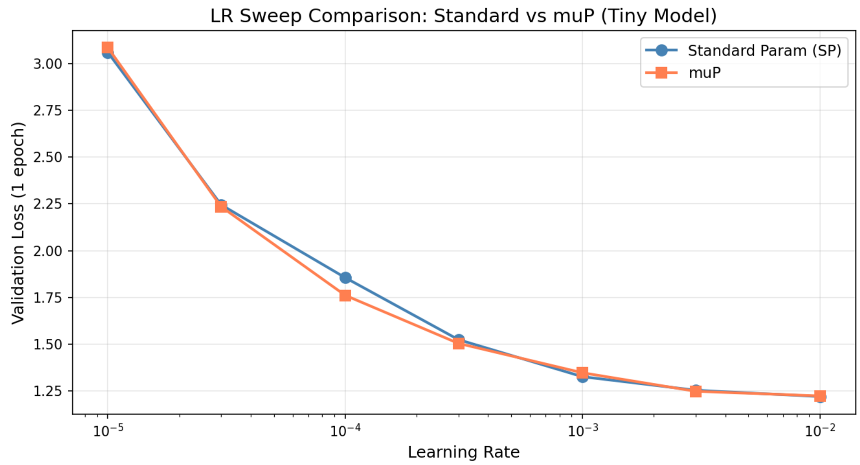


Figure 4: LR sweep on the Tiny model. SP and μP both select 10^{-2}. The μP advantage is not visible at base width; it emerges at scale.

4.3.2 SP vs. μP Scaling

Table 3 and Figure 5 compare validation losses across all five model sizes. The crossover between SP and μP falls between Medium (12M) and Large (33M). For Tiny and Small, the fixed LR of 10^{-2} happens to be close enough to optimal that SP is slightly competitive; for Large and XL it is not, and the gap widens with scale. At the XL model, μP reduces validation loss by 0.392 nats relative to SP (from 1.367 to 0.975), a difference that would require roughly $6\times$ more parameters to recover under SP.

Table 3: SP vs. μP validation loss after 1 epoch.

Model	SP Params	SP Val Loss	μP Val Loss	Improvement
Tiny	1.35M	1.229	1.221	+0.008
Small	3.51M	1.051	1.074	−0.023
Medium	12.32M	0.947	1.025	−0.078
Large	33.75M	1.067	0.941	+0.126
XL	88.40M	1.367	0.975	+0.392

The power law fit to the μP losses is:

$$L_{\mu P}(N) = 72852.3 \cdot N^{-0.866} + 0.952 \qquad (R^2 = 0.960) \tag{5}$$

The scaling exponent $\alpha = 0.866$ means that a 10-fold increase in parameter count reduces the excess loss above the fitted floor by a factor of approximately $10^{0.866} \approx 7.3$. The loss floor itself is non-trivial: even an infinitely large model would not reach zero loss on this corpus, reflecting irreducible variance in SVG structure that no token prediction model can eliminate.

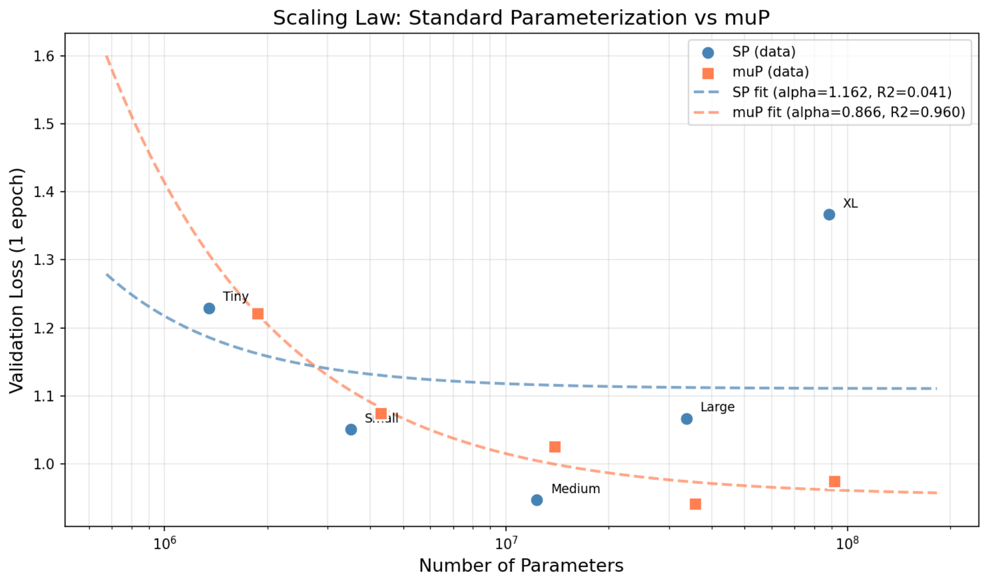


Figure 5: Scaling curves for SP and μP with power law fits. SP fits poorly ($R^2 = 0.041$); μP scales smoothly ($R^2 = 0.960$, $\alpha = 0.866$).

4.3.3 Extrapolation to 10× Scale

Using the μP fit, validation loss is extrapolated to a hypothetical model with 10× the parameter count of the SP XL reference (8.84×10^8 parameters):

$$L_{\mu P}(8.84 \times 10^8) = 72852.3 \cdot (8.84 \times 10^8)^{-0.866} + 0.952 \approx 0.954 \tag{6}$$

A 95% confidence interval computed from the `curve_fit` covariance matrix gives:

$$\hat{L} = 0.954, \quad 95\%\ \text{CI} = [0.888,\ 1.019], \quad \text{SE} = 0.033$$

Figure 6 shows the extrapolation. The interval is narrow in a statistical sense, but it captures only the uncertainty from fitting five data points to a three-parameter model. Practical extrapolation uncertainty is larger for two reasons: the power law may not hold a full order of magnitude beyond the range of the fit, and the analysis varies only N while holding dataset size and compute fixed, which departs from the compute-optimal frontier studied by Hoffmann et al. [2]. This interval should not be interpreted as a true uncertainty bound on the actual loss of a future 884M-parameter model; it only reflects uncertainty under the assumed functional form. The predicted value (0.954) is only 0.021 nats below the XL μP result (0.975), meaning a model ten times larger is predicted to recover less than a tenth of a nat in validation loss. The closeness of the prediction to the loss floor (0.952) makes the conclusion concrete. Further scaling alone, without expanding the dataset, returns negligible improvement at this operating point.

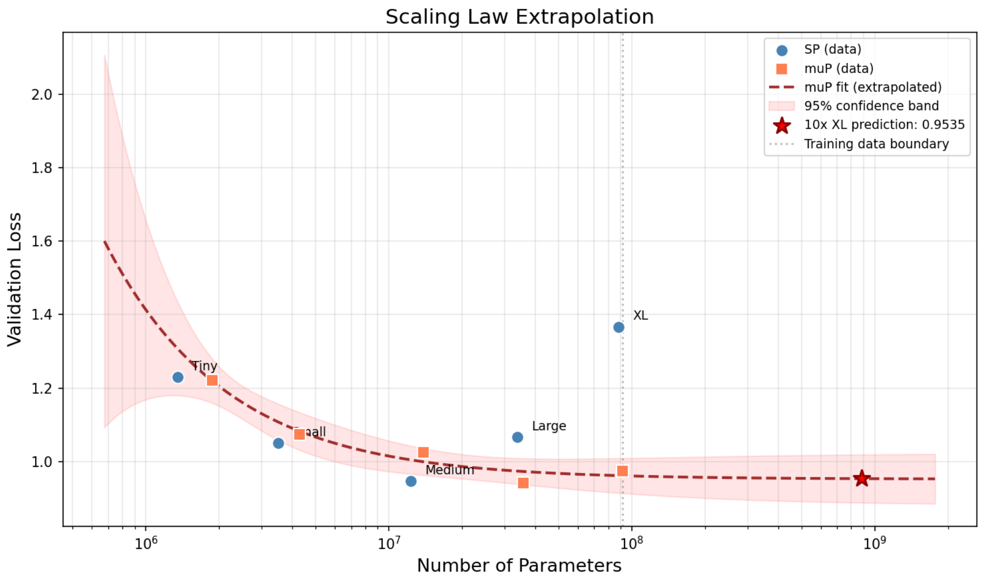


Figure 6: μP power law extrapolated to 8.84×10^8 parameters. The red star marks the predicted loss of 0.954, with the shaded band showing the 95% CI. The dotted line marks the start of extrapolation.

4.4 Best Model Training and SVG Generation

4.4.1 Extended Training

Although the Large μP model achieved the lowest one-epoch validation loss (0.941), the XL μP model is selected for extended training because it has the largest capacity and demonstrated stable training dynamics under μP, unlike its SP counterpart. It is trained for 5 epochs under the same optimizer, schedule, and batch size, with the cosine schedule spanning all $5 \times 7{,}103 = 35{,}515$ steps and the learning rate decaying from 10^{-2} to 10^{-3}. Total wall time is 22,520 seconds (6.25 hours). Final validation loss is 0.920.

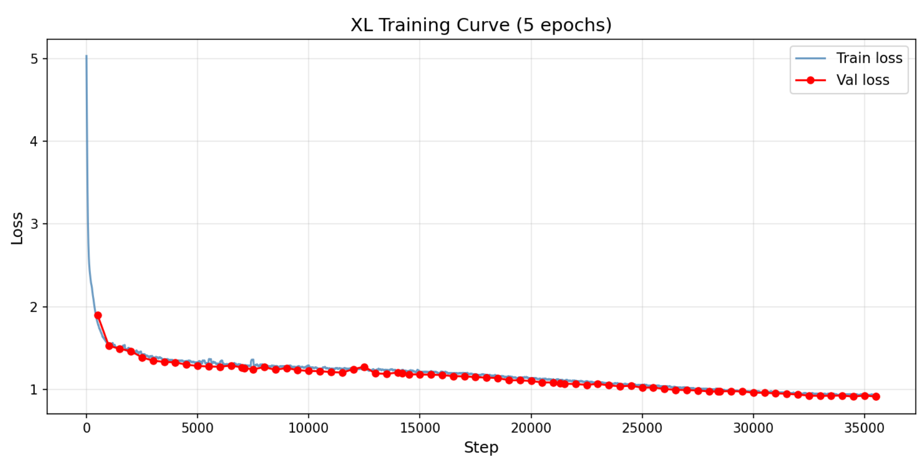


Figure 7: Training and validation loss for the XL μP model over 5 epochs (35,515 steps). Both curves decrease steadily with no sign of overfitting.

On the held-out test set:

$$\mathcal{L}_{\text{test}} = 0.919, \qquad \text{PPL} = \exp(0.919) \approx 2.51$$

4.4.2 Generation

Samples are generated autoregressively from a fixed `<svg ...>` opening tag using combined top-k ($k = 50$) and nucleus (top-$p = 0.95$) sampling. Temperature τ scales the logits before sampling:

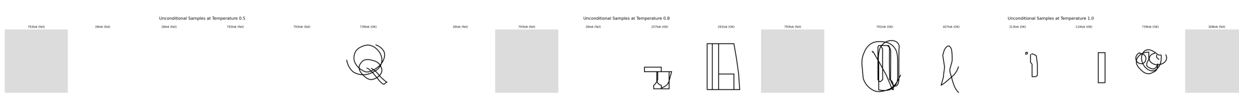
$$p_i = \frac{\exp(z_i/\tau)}{\sum_j \exp(z_j/\tau)} \tag{7}$$

A post-processing repair step handles outputs truncated at the maximum generation length: incomplete element tags are removed, unclosed self-closing elements are closed, `<g>` group tags are balanced, and `</svg>` is appended if absent.

4.4.3 Unconditional Generation

Eighteen unconditional samples are generated at three temperatures ($\tau \in \{0.5, 0.8, 1.0\}$, six samples each). Figure 8 shows the results.

Temperature 1.0 produces the most renderable outputs (5/6) while temperature 0.5 produces the fewest (1/6). This reversal of the expected quality-diversity tradeoff traces to a bimodal collapse at low temperature: outputs at $\tau = 0.5$ are either 26 tokens long (the model predicts end-of-sequence immediately after the SVG opening tag, producing an empty `<svg/>`) or 793 tokens long (the model generates until the maximum length limit with no stopping point), with almost nothing in between. Temperature 0.8 produces the most visually coherent outputs, with clean lines and centered compositions resembling actual icons, but its render rate is lower than $\tau = 1.0$ because more outputs are truncated mid-element. At temperature 1.0, the model generates longer, more varied sequences with actual path and shape elements at the cost of occasional malformed XML and more abstract visual outputs.



(a) $\tau = 0.5$ (b) $\tau = 0.8$ (c) $\tau = 1.0$

Figure 8: Unconditional generation at three temperatures. $\tau = 0.5$ collapses to mostly blank outputs; $\tau = 1.0$ produces the most visible shapes.

4.4.4 Prefix-Conditioned Generation

Five prefix-conditioned samples are generated at $\tau = 0.8$. All five completions parse as valid XML and pass the CairoSVG loose render check. The `star_start` prefix (†) exhausts the 768-token generation limit before closing the polygon; CairoSVG processes the resulting file but the rendered output is a blank canvas, so it fails the strict alpha-channel check. The `group_one_shape` prefix generates only a single closing tag token, suggesting the model interprets the supplied rectangle as a complete composition rather than a starting point.

Among the remaining completions, `two_circles` is the strongest result: the model adds a third circle in a spatially consistent position, demonstrating that it can extend a geometric pattern rather than simply appending unrelated elements. The `partial_face` completion adds a curved mouth but does not place a second eye, showing that the model can continue local structure (face elements) while missing global bilateral symmetry. The `open_path` completion closes the triangle with minimal additional strokes.

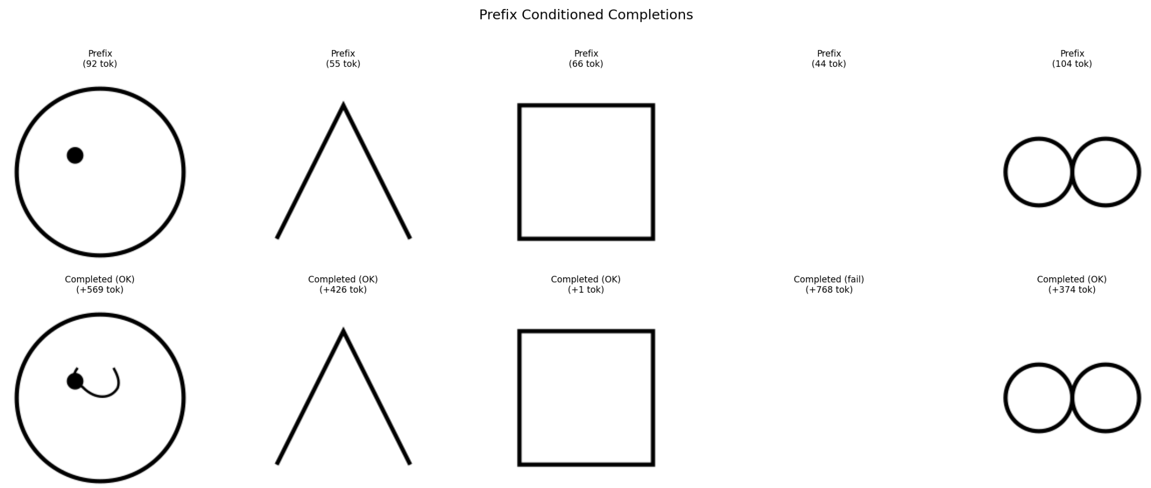


Figure 9: Prefix-conditioned completions. Each column shows the rendered prefix (top) and the model's full completion (bottom).

4.4.5 Quantitative Evaluation

Across all 23 generated samples (18 unconditional + 5 prefix), the model achieves a test perplexity of 2.51, XML validity of 82.6% (19/23), and a loose SVG render rate of 82.6% (52.2% under the stricter alpha-channel render check). Structural validity is high: every sample includes the correct `<svg>` root element with `xmlns` namespace and `viewBox` attribute, all tags are properly closed, and attribute values are well-formed. Path elements appear in 65% of outputs and stroke/fill attributes in 70%, consistent with the icon- and glyph-heavy training corpus. The near-absence of `<g>` group elements (4%) reflects the flat element hierarchies prevalent in the data.

5 Discussion

The central result of this project is that neural scaling laws are not an automatic property of model size. They depend on the training dynamics being well-behaved across scales, which in turn requires that the learning rate be appropriate at each width. Under SP, the LR calibrated on a 1.35M-parameter model overdrives a 88.4M-parameter model, producing a scaling curve that is actively misleading: a naive reading would conclude that more parameters make things worse. μP removes this confound, and the smooth power law that emerges ($R^2 = 0.960$) confirms that the underlying data and architecture do support consistent scaling. The failure was in the parameterization, not the hypothesis.

Several design decisions had meaningful consequences. The choice of a 256-token context window, necessary to keep all five model sizes within GPU memory, means the model is trained exclusively on partial SVGs, since the median training sequence is 475 tokens, nearly twice the context size. This limits what the model can learn about global SVG structure. The generated images show that the model reliably produces correct element syntax and valid attribute values, but it rarely produces SVGs with deliberate spatial symmetry or multi-element compositions, capabilities that likely require attending over a complete SVG during training. A 512- or 1,024-token context would be the clearest path to improving generation quality beyond what larger models alone can provide.

The mode collapse at low temperature is a separate and more interesting failure. At $\tau = 0.5$, the model terminates most outputs immediately after the SVG opening tag, producing syntactically valid but content-free files. This suggests that the EOS token has a high unconditional probability in the model's learned distribution, plausibly because many training SVGs are short and terminate quickly after their opening tag, leading the model to associate the opening context with a high prior for early termination. This is a corpus composition artifact rather than a capacity limitation, and it highlights that generation quality evaluation should not rely on low temperatures alone.

Comparing the scaling exponent $\alpha = 0.866$ to the values reported by Kaplan et al. [1] ($\alpha \approx 0.07$ in loss-vs-compute) requires care. Their exponents are measured along the compute-optimal frontier where N, D, and compute budget are co-varied; ours hold D fixed and vary only N, which produces a steeper slope. The two quantities are not directly comparable. Within the fixed-data regime our experiment occupies, the steep slope is encouraging: it means that at the current dataset size, each doubling of model parameters still returns substantial loss reduction above the floor. The loss floor itself ($c = 0.952$) indicates that a meaningful portion of the variation in SVG structure is not predictable from local token context alone, and represents a lower bound on achievable test loss under this training setup regardless of model size.

6 Conclusion

Five transformer models trained on a 115M-token SVG corpus demonstrate that power-law scaling laws hold for structured visual code, but only when the learning rate is properly transferred across model widths. Standard parameterization with a fixed learning rate produces non-monotonic scaling and a meaningless power law fit ($R^2 = 0.041$). Replacing SP with μP restores consistent scaling ($\alpha = 0.866$, $R^2 = 0.960$) at no additional computational cost. The LR sweep is performed once on the smallest model and transferred without modification to models 66× larger.

The best model, XL μP trained for 5 epochs, achieves a test perplexity of 2.51 and produces valid, renderable SVG in 82.6% of sampled outputs. Qualitative inspection shows that the model has internalized SVG element syntax and attribute conventions. The remaining failure modes (mode collapse at low temperature, limited global compositional structure, and blank-canvas completions at the generation length limit) point to the context window constraint (256 tokens vs. a 475-token median SVG) as the primary bottleneck. Extending the context to 1,024 tokens with a model at this scale, or co-scaling the dataset with model size along the Chinchilla-optimal frontier [2], are the most direct paths to further improvement.

References

[1] J. Kaplan, S. McCandlish, T. Henighan, T. B. Brown, B. Chess, R. Child, S. Gray, A. Radford, J. Wu, and D. Amodei, “Scaling Laws for Neural Language Models,” *arXiv preprint arXiv:2001.08361*, 2020.

[2] J. Hoffmann, S. Borgeaud, A. Mensch, E. Buchatskaya, T. Cai, E. Rutherford, D. de Las Casas, L. A. Hendricks, J. Welbl, A. Clark, et al., "Training Compute-Optimal Large Language Models," *arXiv preprint arXiv:2203.15556*, 2022.

[3] G. Yang, E. J. Hu, I. Babuschkin, S. Sidor, X. Liu, D. Farhi, N. Ryder, J. Pachocki, W. Chen, and J. Gao, "Tensor Programs V: Tuning Large Neural Networks via Zero-Shot Hyperparameter Transfer," *arXiv preprint arXiv:2203.09789*, 2022.

[4] J. C. Rodriguez, P. Lezama, S. Mahendran, J. Delpino, J. Vargas, and G. Sapiro, "StarVector: Generating Scalable Vector Graphics Code from Images and Text," *arXiv preprint arXiv:2312.11556*, 2023.

[5] A. Karpathy, "nanoGPT," https://github.com/karpathy/nanoGPT, 2022.

[6] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, et al., "PyTorch: An Imperative Style, High-Performance Deep Learning Library," *Advances in Neural Information Processing Systems*, 32, 2019.

[7] HuggingFace, "Tokenizers," https://github.com/huggingface/tokenizers, 2020.

[8] lxml Development Team, "lxml: XML and HTML with Python," https://lxml.de, 2024.

[9] CairoSVG Development Team, "CairoSVG," https://cairosvg.org, 2024.

Appendices

A Hyperparameter Summary

Table 4: Full hyperparameter summary for all experiments.

Hyperparameter	Value
Tokenizer	BPE, byte-level, vocab size 4,096
Context window (block size)	256 tokens
Batch size	64 sequences (16,384 tokens/step)
Optimizer	AdamW
Adam betas	(0.9, 0.95)
Weight decay	0.1
Gradient clip norm	1.0
LR schedule	Cosine with 5% linear warmup
Minimum LR	$0.1 \times \eta_{\max}$
Optimal LR (SP and μP)	10^{-2}
Dropout	0.1
μP base width	128
Extended training epochs (Part 4)	5
Generation max tokens	768
Generation top-k	50
Generation top-p (nucleus)	0.95
Hardware	NVIDIA A100-SXM4-40GB

B Extended Training Results

Table 5: Learning rate sweep on the Tiny SP model (1 epoch). Best entry marked with *.

Learning Rate	SP Val Loss	μP Val Loss
1×10^{-5}	3.058	3.084
3×10^{-5}	2.245	2.235
1×10^{-4}	1.856	1.762
3×10^{-4}	1.525	1.505
1×10^{-3}	1.327	1.348
3×10^{-3}	1.255	1.249
1×10^{-2}	**1.221***	**1.225***

Table 6: Training throughput and GPU memory for all SP models (1 epoch).

Model	**Wall Time (s)**	**Tokens/s**	**Peak GPU (MB)**
Tiny	200	580,939	2,286
Small	413	282,007	3,773
Medium	817	142,500	5,152
Large	1,951	59,643	10,136
XL	4,492	25,906	17,687

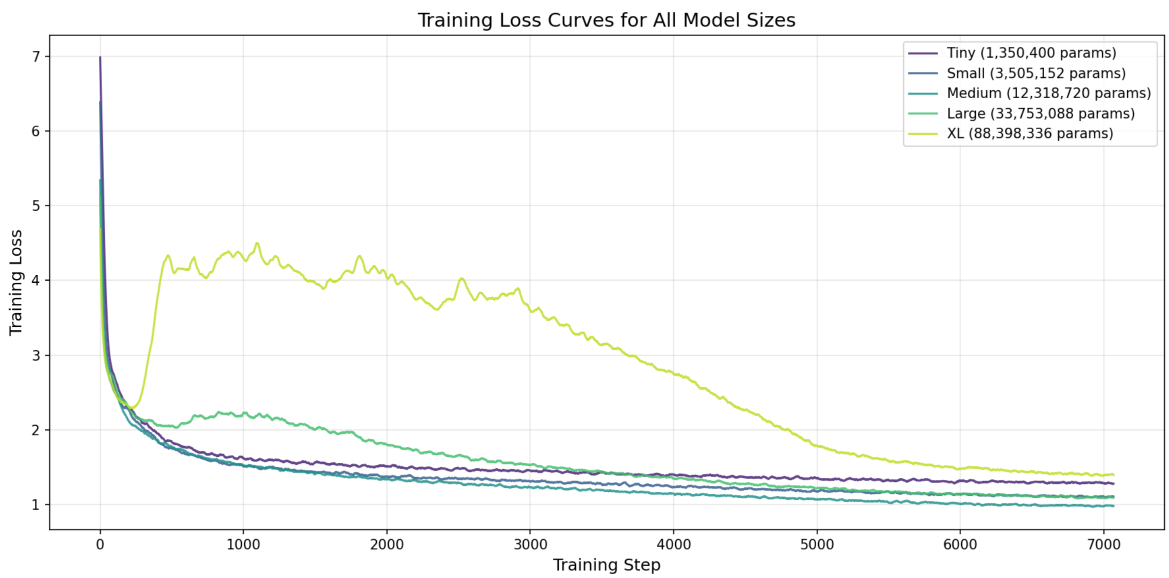


Figure 10: Training loss curves for all five SP models.

Figure 11: Training loss curves for all five μP models.

C SVG Code Example

Listing 1: SVG source for the P5 dataset example (211 tokens, normalized coordinates).

```
<svg xmlns="http://www.w3.org/2000/svg"
     viewBox="0.0␣0.0␣24.0␣24.0"
     height="200px" width="200px">
  <path fill="none" stroke="black"
        stroke-width=".3" stroke-opacity="1.0"
        d="M16.5␣7.8␣L14.9␣15.0␣..."/>
</svg>
```